

献血管理系统项目总结报告

1. 项目概述

本项目实现一套桌面端献血管理系统，将用户档案、血液库存批次、用血申请、评论评分、统计报表和操作审计整合到统一界面中。

系统使用 MySQL 保存具有明确关系和事务要求的核心数据，使用 MongoDB 保存结构灵活、写入频繁或需要聚合统计的扩展数据。两类数据库通过 MySQL 业务 ID 建立关联。

2. 项目目标完成情况

目标	完成情况
用户注册和登录	已完成，支持 BCrypt 密码哈希和登录日志
用户与档案管理	已完成，支持联系方式、实名档案、角色和状态
分类管理	已完成，支持父子分类和循环关系校验
血液库存管理	已完成，支持基本信息、详情、血型、状态和数量
用血申请	已完成，支持创建、编辑、审批、取消和删除
库存扣减事务	已完成，使用行锁、显式提交和回滚
评论和评分	已完成，支持内容、评分、标签和删除权限
统计报表	已完成，包含月度用血、热度、评分、用户行为和审计
跨数据库展示	已完成，合并库存、订单、详情、评论和行为数据
库存推荐	已完成，结合用户记录、分类、热度和评分
日志审计	已完成，包含登录日志、行为日志和审计汇总
自动化测试	已完成，常规测试和 MongoDB 压力测试通过

3. 数据库设计总结

3.1 MySQL

MySQL 保存以下核心数据：

- `users`：用户、密码哈希、角色和状态。
- `categories`：父子层级分类。

- `items`：血液库存批次和可用数量。
- `orders`：用血申请和处理状态。
- `profiles`：用户或献血者实名档案。

数据库使用主键、外键、唯一约束、检查约束和索引保证数据关系和查询效率。视图用于用户档案、库存汇总、待审批申请和月度汇总；存储过程用于报表；触发器用于数量和状态的兜底校验。

3.2 MongoDB

MongoDB 保存以下扩展数据：

- `item_details`：描述、图片和血型等扩展属性。
- `comments`：评论、评分和标签。
- `action_logs`：用户行为、客户端信息和操作时间。
- `system_logs`：登录、警告、错误和关键操作日志。

集合根据用户、批次、类型和时间建立索引。热度、评分和审计统计使用聚合管道完成。

4. 核心业务实现

4.1 用户认证

用户注册时检查用户名、邮箱和手机号，密码使用 BCrypt 哈希保存。登录时检查账号状态和密码，并将成功或失败结果写入系统日志。

4.2 用血申请

普通用户选择可用库存并填写数量后创建待审批申请。系统在创建前检查库存状态和数量。普通用户只能维护自己的待审批记录。

4.3 审批和库存扣减

管理员审批申请时，系统在 MySQL 事务中锁定申请和库存记录，重新检查状态和数量，然后更新申请状态并扣减库存。任何一步失败都会回滚。

这一流程避免了两个审批操作同时读取旧库存数量并重复扣减的问题。

4.4 统计和推荐

报表数据来自 MySQL 存储过程和 MongoDB 聚合管道。库存推荐优先考虑用户申请或访问过的分类，再根据行为热度和评分排序，并为每条推荐返回可理解的理由。

5. 安全和可追溯性

项目已采用以下措施：

- BCrypt 密码哈希。
- PreparedStatement 参数绑定。
- 账号启用和停用状态。
- 管理员与普通用户的功能隔离。
- 对用户档案、订单和评论执行归属权检查。
- 保护超级管理员和当前登录账号。
- 登录日志、用户行为日志和操作审计。
- 主外键、唯一约束、检查约束和触发器。

演示数据使用 `.test` 邮箱、测试证件号和测试地址，不包含真实个人敏感信息。

6. 测试结果总结

6.1 常规自动化测试

```
Tests run: 50, Failures: 0, Errors: 0, Skipped: 1
BUILD SUCCESS
```

默认跳过的用例是需要 MongoDB 的压力测试。其余 49 项测试全部通过。

6.2 MongoDB 压力测试

指标	结果
日志写入数	10000 条
并发数	50
耗时	2.547 秒
吞吐量	3926.19 条/秒
数量校验	通过
执行结果	BUILD SUCCESS

压力测试满足课程要求的 10000 条日志和 50 并发规模。

7. 项目中的主要难点

7.1 混合数据库边界

核心问题不是同时使用两种数据库，而是确定哪些数据必须依赖关系约束和事务，哪些数据需要更灵活的文档结构和聚合能力。

项目将库存、申请和用户关系保存在 MySQL，将详情、评论和日志保存在 MongoDB，避免把 MongoDB 简单当作另一个关系数据库。

7.2 库存并发一致性

审批时不能只在界面或服务层读取一次库存后直接扣减，因为并发操作可能使读取到的数量失效。项目在事务内重新查询并锁定记录，保证检查和扣减使用同一个一致视图。

7.3 角色与数据范围

权限不只是隐藏界面按钮。服务层同时检查当前用户、目标用户和管理员状态，防止普通用户通过非界面路径修改他人记录。

8. 项目收获

通过项目实践，完成以下能力训练：

- 将业务需求转换为关系表、约束和索引。
- 根据数据特点选择关系模型或文档模型。
- 使用 JDBC、PreparedStatement、连接池和显式事务。
- 使用 MongoDB 索引、嵌套文档、数组和聚合管道。
- 在服务层表达权限、状态和数量等业务规则。
- 使用自动化测试保护核心流程和回归问题。
- 使用可重复的压力测试验证并发写入。
- 编写需求、设计、测试、用户操作和答辩材料。

9. 已知边界

9.1 桌面应用范围

系统为课程演示用 Swing 桌面应用，适用于单机演示和小规模业务操作，不包含 Web 部署、移动端和外部医院系统接口。

9.2 跨数据库一致性

MySQL 和 MongoDB 之间不使用分布式事务。MySQL 保证核心业务数据，MongoDB 扩展数据在写入失败时可能需要补写。如项目扩展为长期运行系统，可增加对账和失败重试机制。

9.3 安全加固

当前数据库凭据是课程 Docker 环境的本地配置。公开部署前应使用环境变量或密钥管理，并增加密码强度要求和登录失败限流。

10. 后续改进方向

如课程外继续扩展，可按实际需求选择以下方向：

- 1. 增加数据库连接故障提示和有限重试。
- 2. 将数据库凭据移出版本库。
- 3. 增加密码强度和登录失败限制。
- 4. 为跨数据库扩展数据增加补写和对账。
- 5. 根据真实数据量增加分页查询和更完整的查询性能基准。

这些方向不影响当前课程验收，应在出现明确使用场景后再实施。

11. 交付物总结

交付物	状态
Git 项目源码	已完成
需求规格说明书	已完成 Markdown 版
MySQL E-R 图	已完成
MongoDB 数据库设计	已完成
MySQL DDL 和配套脚本	已完成
MongoDB 初始化和演示数据脚本	已完成
性能优化报告和安全清单	已完成 Markdown 版
测试报告	已完成 Markdown 版
技术设计文档	已完成 PDF 版
用户手册	已完成 Markdown 版
答辩预演记录	已完成 Markdown 版
代码审查自检清单	已完成 Markdown 版

交付物	状态
项目总结报告	已完成 Markdown 版
答辩 PPT	不在本次补充范围

12. 总结

项目完成了从需求分析、数据模型设计、数据库脚本、Java 数据访问、业务事务、桌面界面到测试和交付文档的完整流程。

核心业务数据由 MySQL 保证关系和事务一致性，扩展详情和日志由 MongoDB 提供灵活存储和聚合能力。系统具备可演示的用户、库存、申请、评论、报表和审计流程，并通过自动化测试和压力测试验证。

当前成果满足课程项目的主要功能、数据库设计、技术深度、测试和文档交付要求。